
FILES MODULES IN ANSIBLE

1. Introduction

Red Hat Ansible Engine includes a large collection of modules known as the **module library**.

To simplify organization and documentation, modules are grouped by function.

The **Files Modules Library** contains modules used for:

- Creating files
- Copying files
- Editing files
- Removing files
- Managing permissions
- Managing SELinux contexts
- Synchronizing directories

These modules automate common Linux file-management tasks across managed hosts.

2. Commonly Used Files Modules

Table 1: Frequently Used File Management Modules

Module Name	Description
blockinfile	Insert, update, or remove a block of multiline text with customizable markers.
copy	Copy files from control node → managed hosts. Can set permissions and SELinux context.
fetch	Reverse of copy. Fetch files from managed hosts → control node.
file	Manage file attributes (permissions, owner, SELinux type). Create/remove files, dirs, links.

Module Name	Description
lineinfile	Ensure a specific line exists or replace it using regex.
stat	Retrieve file metadata similar to Linux stat.
synchronize	Wrapper for rsync to sync files/directories efficiently.

3. Automation Examples Using Files Modules

3.1 Ensuring a File Exists on Managed Hosts

Example: Touch a File and Set Permissions

- name: Touch a file and set permissions

file:

path: /path/to/file

owner: user1

group: group1

mode: '0640'

state: touch

Example Outcome

\$ ls -l file

-rw-r----- 1 user1 group1 0 Nov 25 08:00 file

3.2 Modifying File Attributes

Scenario:

A file in a user's home directory has the wrong SELinux context:

```
$ ls -Z samba_file
```

```
-rw-r--r-- owner group unconfined_u:object_r:user_home_t:s0 samba_file
```

Fix SELinux Type Using file Module

- name: SELinux type is set to samba_share_t

file:

path: /path/to/samba_file

setype: samba_share_t

Outcome

```
$ ls -Z samba_file
```

```
-rw-r--r-- owner group unconfined_u:object_r:samba_share_t:s0 samba_file
```

3.3 Making SELinux File Context Changes Persistent

Problem:

file: module behaves like chcon — changes are **not persistent** and can be undone by restorecon.

Solution: Use sefcontext to update SELinux policy.

- name: SELinux type is persistently set to samba_share_t

sefcontext:

target: /path/to/samba_file

setype: samba_share_t

state: present

IMPORTANT

The **sefcontext** module updates the SELinux policy default context but does *not* relabel existing files.

You must run **restorecon** to apply the new context.

sefcontext only updates the SELinux *policy* default file context.
It does **NOT** relabel existing files.

This means:

- It writes rules into:
/etc/selinux/targeted/contexts/files/file_contexts.local
- It tells SELinux:
"From now on, files matching this path pattern *should* have this context."
- But it does **not** touch any files already on disk
- Because relabeling can be:
 - slow
 - disruptive
 - dangerous if applied to the wrong path
 - something admins want explicit control over

3.4 Copying Files to Managed Hosts

Example: Copy a File

- name: Copy a file to managed hosts

copy:

src: file

dest: /path/to/file

Notes:

- Default behavior: force: yes → overwrite if contents differ
- Set force: no to copy only if file does not exist

3.5 Retrieving Files from Managed Hosts

Example: Fetch SSH Public Key

- name: Retrieve SSH key from reference host

fetch:

src: "/home/{{ user }}/ssh/id_rsa.pub"

dest: "files/keys/{{ user }}.pub"

3.6 Editing Files on Managed Hosts

Add a Single Line Using lineinfile

- name: Add a line of text to a file

lineinfile:

path: /path/to/file

line: 'Add this line to the file'

state: present

Add a Block of Text Using blockinfile

- name: Add additional lines to a file

blockinfile:

path: /path/to/file

block: |

First line in the additional block of text

Second line in the additional block of text

state: present

Resulting File Content

```
# BEGIN ANSIBLE MANAGED BLOCK
```

```
First line in the additional block of text
```

```
Second line in the additional block of text
```

```
# END ANSIBLE MANAGED BLOCK
```

3.7 Removing a File

```
- name: Make sure a file does not exist on managed hosts
```

```
file:
```

```
  dest: /path/to/file
```

```
  state: absent
```

3.8 Retrieving File Status Using stat

Example: Get MD5 Checksum

```
- name: Verify the checksum of a file
```

```
stat:
```

```
  path: /path/to/file
```

```
  checksum_algorithm: md5
```

```
  register: result
```

```
- debug:
```

```
  msg: "The checksum of the file is {{ result.stat.checksum }}"
```

Example Output

```
"msg": "The checksum of the file is 5f76590425303022e933c43a7f2092a3"
```

3.9 Display All stat Output

- name: stat /etc/passwd

stat:

path: /etc/passwd

register: results

- name: Display stat results

debug:

var: results

3.10 Synchronizing Files Using synchronize

What the synchronize Module Does

The module simplifies common rsync operations:

- Push files from control node → managed host
- Pull files from managed host → control node
- Mirror directories (including delete-missing)
- Sync large datasets efficiently using rsync delta algorithm
- Preserve permissions, ownership, timestamps
- Use SSH transport automatically

It is ideal for backups, web content deployment, log collection, and large directory replication.

Key Parameters (Most Useful in Real Work)

- **src** — Source path
- **dest** — Destination path
- **mode** — push (default) or pull
- **recursive** — Sync directories recursively
- **delete** — Remove files on destination that don't exist on source

- **compress** — Use compression during transfer
- **rsync_opts** — Add custom rsync options
- **archive** — Preserve permissions, ownership, timestamps

How It Works Internally

- It does not run on the remote host like other modules.
- It runs rsync on the control node, using SSH to connect to the managed host.
- For Windows hosts, it does not work unless rsync is installed manually.
-

Common Usage Examples

1 **Push a directory to a managed host**

```
```yaml
```

```
- name: Sync website content to web server
```

```
 synchronize:
```

```
 src: /var/www/html/
```

```
 dest: /var/www/html/
```

```
 recursive: yes
```

```
 delete: yes
```

```
```
```

2 **Pull logs from a managed host to control node**

```
```yaml
```

```
- name: Collect logs from remote server
```

```
 synchronize:
```

```
 mode: pull
```

```
 src: /var/log/
```

```
 dest: /backup/logs/{{ inventory_hostname }}/
```

```
```
```


****Sync with compression + archive mode****

```
```yaml
```

```
- name: Efficient sync with compression
```

```
 synchronize:
```

```
 src: /data/
```

```
 dest: /backup/data/
```

```
 archive: yes
```

```
 compress: yes
```

```
```
```

****Add custom rsync options****

```
```yaml
```

```
- name: Sync excluding temporary files
```

```
 synchronize:
```

```
 src: /app/
```

```
 dest: /deploy/app/
```

```
 rsync_opts:
```

```
 - "--exclude=*.tmp"
```

```
 - "--exclude=cache/"
```

```
```
```

Notes:

- Requires rsync installed on both ends
 - Supports directory sync, excludes, recursive options
-

4. Summary

Files Modules Provide Automation For:

- Creating files
- Editing files
- Copying files
- Removing files
- Managing SELinux contexts
- Retrieving file metadata
- Synchronizing directories

These modules are essential for RHCSA/RHCE automation tasks.

